

LC 1091 - Shortest Path in Binary Matrix

Samsung

JPMC

Goldman

Dunzo

MMT

Sprinklr

Problem Definitions

- **0** = free cell
- **1** = blocked cell
- **start** = (0, 0)
- **end** = (n-1), (n-1)
- **8 direction**
- If impossible → return -1
- Otherwise (min path)

Core Concept

whenever you see Shortest Path, min moves, min operation → BFS

because DFS wanders, BFS goes distance by distance the first time we reach destination we get shortest path

Observations

Obv - 1

If start = block or end = block → ret -1

Obv - 2

to mark a cell as visited mark it as 1 → i.e block it for further traversal

Obv - 3

whenever $r == n-1$ && $c == n-1$ → return distance


```

class Solution {
public:
    int shortestPathBinaryMatrix(vector<vector<int>>& grid) {

        int n=grid.size();
        int m=grid[0].size();

        vector<pair<int,int>> dir={{1,0},{-1,0},{0,0},{-1,-1},{1,-1},{0,-1},{1,1},{0,1}};
        queue<pair<int,int>> q;

        if(grid[0][0]==1 || grid[n-1][m-1]==1){
            return -1;
        }

        grid[0][0]=1;
        q.push({0,0});
        int ans=1;

        while(!q.empty()){

            int currsz=q.size();
            while(currsz--){
                int r=q.front().first;
                int c=q.front().second;
                if(r==n-1 && c==m-1) return ans;
                q.pop();

                for(auto it:dir){
                    int nr=r+it.first;
                    int nc=c+it.second;

                    if(nr>=0 && nc>=0 && nr<n && nc<m){
                        if(grid[nr][nc]==0){
                            grid[nr][nc]=1;
                            q.push({nr,nc});
                        }
                    }
                }
            }

            ans++;

        }

        return -1;
    }
};

```

```
}  
};
```